

TNC Lightbox — Campaign setup guide

A small popup ("lightbox") you can add to any Engaging Networks page. You control what it says, when it appears, and how it looks — all from a few lines of config.

Table of contents

- [How it works](#)
 - [Adding it to a page](#)
 - [When it appears — the triggers](#)
 - [Designs work with any trigger](#)
 - [Linking the button to another page](#)
 - [Design options](#)
 - [Themes \(the overall look\)](#)
 - [Eyebrow label](#)
 - [One button \(recommended\)](#)
 - [Image position](#)
 - [Close button](#)
 - [How often it shows](#)
 - [Analytics tracking](#)
 - [Optional diagnostics](#)
 - [Real examples](#)
 - [Example 1: Exit-intent monthly-giving nudge \(forest\)](#)
 - [Example 2: Time-delayed donation prompt \(sky\)](#)
 - [Example 3: Scroll-triggered petition signup](#)
 - [Example 4: Exit-intent redirect to a donation form \(records the outcome on both pages\)](#)
 - [FAQ](#)
-

How it works

The lightbox is a small dialog that appears on top of the page. It never blocks the page from loading and never interferes with an Engaging Networks donation form. You configure it once, paste two snippets onto your page, and it handles the rest — waits for the trigger you set, respects a "don't nag" frequency, and remembers when someone has already seen it.

Adding it to a page

Two things go on the page: a **config block** (your settings) and a **script tag** (the code that runs it). Place them anywhere — the `<head>` or before `</body>` both work.

```
<script>
  window.ENLightbox = {
    header: "Join the fight",
    body: "Add your voice to protect the lands and waters we all rely on.",
    cta: { label: "Sign now", href: "#petition", action: "redirect" },
  };
</script>
<script src="https://aaf1a18515da0e792f78-c27fdabe952dfc357fe25ebf5c8897ee.ssl.cf5.rackcdn.com/2246/en-lightbox.js" async></script>
```

The config must be set **before or at the same time** as the script tag. The popup does **not** appear immediately — it waits for the trigger you configure.

When it appears — the triggers

You choose what makes the popup show up. You can use one trigger or combine several ("first one to fire wins").

Trigger	What it does	Good for	
<code>time</code>	Appears after a set number of seconds	Letting people read the page first, then showing a prompt	
<code>scroll</code>	Appears when the visitor has scrolled a certain percentage down the page	Catching engaged readers who are already interested	
<code>inactivity</code>	Appears after a few seconds of mouse/touch inactivity	Getting attention from someone who stopped interacting	
<code>exitIntent</code>	Appears when the mouse moves toward the browser's close button or address bar — desktop only	A last chance before someone leaves	

Combine triggers — first one wins

```
window.ENLightbox = {
  header: "Before you go",
  body: "Will you sign the petition to protect our rivers?",
  cta: { label: "Sign now", action: "close" },
  dismissLabel: "Not now",
  triggers: {
    frequencyDays: 7,
    list: [
      { type: "time", delayMs: 30000 }, // 30 seconds
      { type: "exit-intent" },          // or when mouse leaves
    ],
  },
};
```

Whichever trigger fires first opens the popup, and the others are cancelled.

Designs work with any trigger

The **look** (theme, layout, colors) and the **trigger** are independent. You can use any design with any trigger — a gentle forest-themed popup on a timer, a bright sky-themed popup on exit, a dark popup on scroll. Nothing is locked together.

Linking the button to another page

The button in the popup can either **navigate to a new page** (like a donation form) or **just close** the popup.

```
// Navigate to a donation page
cta: { label: "Donate monthly", href: "https://support.nature.org/...", action: "redirect"
}

// Just close the popup (no page navigation)
cta: { label: "Got it", action: "close" }
```

When you provide an `href` and set `action: "redirect"`, the button becomes a real link — visitors can middle-click, right-click, or open it in a new tab like any normal link. If you leave out `action` entirely but include an `href`, it defaults to `"redirect"`.

This works on every version of the popup, including the exit-intent one.

Design options

Themes (the overall look)

Five preset themes — **forest** and **sky** match the mockups you've seen, alongside `light`, `dark`, and `brand`. All five use the same campaign-style 50/50 layout (half image, half content). You control which side the image sits on with `imagePosition` (see below) — the theme does not fix it. As a convention, `forest` pairs with the image on the right and `sky` with the image on the left to match the mockups.

Theme	What it looks like
<code>forest</code>	Deep green panel (<code>#006537</code>), white body text, a white button with green text, and a square green × close button over the image. Centered, bold campaign text. (Pair with <code>imagePosition: "right"</code> .)
<code>sky</code>	Light blue panel (<code>#8DBBDC</code>), near-black text (<code>#191919</code>), a black button with white text, and a plain black × close icon (no box) over the content. Centered, bold campaign text. (Pair with <code>imagePosition: "left"</code> .)
<code>light</code>	Clean white panel, dark text, blue button — a simple default.
<code>dark</code>	Dark gray panel, light text, a white button with dark text — good for low-light pages.
<code>brand</code>	TNC green (<code>#003d24</code>) panel, white text, green button — the classic TNC look.

On wide screens the modal is about 835×475px; below about 700px the two halves stack vertically and the modal shrinks to fit. When stacked, the **image always appears on top** (regardless of your `imagePosition` setting — the desktop side-by-side order is left alone), and the **close × always has a visible backing** so it stays readable over the image. By default the image still shows on mobile — set `hideImageOnMobile: true` to hide it. Every theme's colors can be tweaked individually — just add a `colors` block:

```
theme: { preset: "forest", colors: { ctaBg: "#004d2e" } }
```

Eyebrow label

An optional small uppercase label that sits above the headline. Use it to add urgency or context.

```
eyebrow: "Limited time"
```

Rendered as: **LIMITED TIME** (small, uppercase, bold) above the main heading.

One button (recommended)

For the forest and sky themes, we recommend using a single call-to-action button and no secondary link — it keeps the design clean and focused. Just set `cta` and leave out `secondaryCta` and `dismissLabel`.

```
cta: { label: "Donate monthly", action: "close" }  
// No secondaryCta, no dismissLabel
```

Image position

Control where the image sits relative to the text.

```
layout: { imagePosition: "left" } // image on the left (default)  
layout: { imagePosition: "right" } // image on the right  
layout: { imagePosition: "top" } // image above the text
```

This sets the image side for **every** theme (the theme does not fix the column order). As a convention, `forest` looks best with `imagePosition: "right"` (content left / image right) and `sky` with `imagePosition: "left"` (image left / content right), matching the mockups — but you can use any position with any theme.

Close button

The close button (x) is larger and easier to see — 44×44 pixels with a contrasting round backing so it's visible over photographs and colored panels (`light` / `dark` / `brand`). The `forest` and `sky` campaign themes have their own close x instead: `forest` uses a square green button with a white x over the image; `sky` uses a plain black x with no backing box over the content panel. On mobile (below ~700px) every theme's close x keeps a visible backing — `sky` gains a semi-opaque dark backing with a white x so it stays readable over the stacked image.

Control its position:

```
layout: { closeButton: "inside" } // top-right corner of the dialog (default)  
layout: { closeButton: "outside" } // above the dialog, not clipped  
layout: { closeButton: "none" } // no close button (rely on CTA, ESC, or backdrop click)
```

How often it shows

Set `triggers.frequencyDays` to control how often the same visitor sees the popup on the same page.

```
triggers: { frequencyDays: 7 } // show at most once per week (default)
triggers: { frequencyDays: 0 } // show on every page load
```

The popup remembers using `localStorage` — even if someone refreshes the page, they won't see it again until the window has passed.

Analytics tracking

Analytics tracking is automatic. There is nothing to add to your lightbox config and no switch to turn on. The page must already have Tealium (`window.utag`), TNC's standard tag manager. When Tealium's `link` function is available, the lightbox sends the events below. When Tealium is not on the page, the lightbox sends nothing and keeps working normally.

What you will see: Your config has no `analytics` setting. You use the same config and script tag as usual; tracking starts when Tealium is available.

These are the only two analytics events:

Plain-language moment	Exact value sent	
The lightbox opens	<code>lightbox_impression</code>	
The visitor clicks the main button (the primary CTA)	<code>lightbox_click</code>	

The impression is sent once each time the lightbox opens. The click is sent only for the main button, including a main button that only closes the lightbox. Closing with X, Escape, by clicking the background, the "no thanks" link, or a secondary button sends no analytics event.

`lightbox_name` is always the fixed value `inactivity-exit` on both events. It stays the same whether the lightbox opened because of time, scroll, inactivity, or exit intent. It is a label for the lightbox, not a third event.

A payload is the small set of values sent for an event.

The two exact payloads are:

```
{ event_name: "lightbox_impression", lightbox_name: "inactivity-exit" }
{ event_name: "lightbox_click", lightbox_name: "inactivity-exit" }
```

No personal or page data is ever included. The payload contains only the two fields shown above.

To verify tracking, add `?debug=true` to the page URL and open the browser console. Open the lightbox and, if appropriate, click its main button. With Tealium on the page, the lines look like this:

```
[ENLightbox debug] utag payload: { event_name: "lightbox_impression", lightbox_name: "inactivity-exit" }  
[ENLightbox debug] utag payload: { event_name: "lightbox_click", lightbox_name: "inactivity-exit" }
```

Without Tealium, the lines use this form instead:

```
[ENLightbox debug] utag absent – would fire: { event_name: "lightbox_impression", lightbox_name: "inactivity-exit" }  
[ENLightbox debug] utag absent – would fire: { event_name: "lightbox_click", lightbox_name: "inactivity-exit" }
```

For the other diagnostic rules, see [Optional diagnostics](#).

Optional diagnostics

For a QA run, append `?debug=true` or `?debug=log` to the **page URL**. Either value turns on console-only diagnostics; it is not a lightbox config setting. The console records lifecycle events (`enlb:open`, `enlb:cta` with its role, and `enlb:dismiss` with its reason and pathname) plus `enlb:field-write` entries for reference-field `write` and `replay` actions. It also records the exact Tealium payload objects for impression and primary-CTA events. If `utag` is absent, QA mode prints `utag absent – would fire: {payload}` instead; diagnostics never queue or send an event.

Without the query, or with any other value such as `debug=false`, `debug=1`, an arbitrary value, or a malformed query, the lightbox produces **zero console output**. Diagnostics stay console-only; an in-page panel is deferred to BACKLOG.

Real examples

Example 1: Exit-intent monthly-giving nudge (forest)

A last-chance popup that appears when someone tries to leave. Uses the forest theme with the image on the right (`imagePosition: "right"`). A single button closes the popup (no page navigation).

```

<script>
  window.ENLightbox = {
    eyebrow: "Last chance",
    header: "Don't go yet",
    body: "Your monthly gift protects forests, rivers, and wildlife year-round.",
    image: { src: "/img/forest-hero.jpg", alt: "Forest landscape" },
    cta: { label: "Give monthly", action: "close" },
    theme: { preset: "forest" },
    layout: { imagePosition: "right" },
    triggers: {
      frequencyDays: 14,
      list: [{ type: "exit-intent" }],
    },
  };
</script>
<script src="https://aaf1a18515da0e792f78-c27fdabe952dfc357fe25ebf5c8897ee.ssl.cf5.rackcdn.com/2246/en-lightbox.js" async></script>

```

Example 2: Time-delayed donation prompt (sky)

Appears after 15 seconds — enough time to read the page. The button links directly to the donation form. Uses the sky theme with the image on the left (`imagePosition: "left"`).


```

<script>
  window.ENLightbox = {
    eyebrow: "Matching gift",
    header: "Double your impact",
    body: "Every dollar donated today is matched through midnight. Don't miss this chance.",
    image: { src: "/img/matching-gift.jpg", alt: "Matching gift" },
    cta: {
      label: "Donate now",
      href: "https://support.nature.org/donate",
      action: "redirect",
    },
    theme: { preset: "sky" },
    layout: { imagePosition: "left" },
    triggers: {
      frequencyDays: 7,
      list: [{ type: "time", delayMs: 15000 }],
    },
  };
</script>
<script src="https://aaf1a18515da0e792f78-c27fdabe952dfc357fe25ebf5c8897ee.ssl.cf5.rackcdn.com/2246/en-lightbox.js" async></script>

```

Example 3: Scroll-triggered petition signup

Appears after the visitor has scrolled 60% down the page — they're already engaged. Links to the petition section. Light theme, image on top, close button placed outside the dialog.

```
<script>
  window.ENLightbox = {
    header: "Add your voice",
    body: "Tell your representatives to protect clean water for generations to come.",
    image: { src: "/img/river.jpg", alt: "Clean river" },
    cta: {
      label: "Sign the petition",
      href: "#petition",
      action: "redirect",
    },
    dismissLabel: "Not right now",
    layout: { imagePosition: "top", closeButton: "outside" },
    triggers: {
      frequencyDays: 30,
      list: [{ type: "scroll", percent: 60 }],
    },
  };
</script>
<script src="https://aaf1a18515da0e792f78-c27fdabe952dfc357fe25ebf5c8897ee.ssl.cf5.rackcdn.com/2246/en-lightbox.js" async></script>
```

Example 4: Exit-intent redirect to a donation form (records the outcome on both pages)

This example uses the same reference field on a cultivation page and its donation form.

Page 1 — cultivation page

```

<script>
  window.ENLightbox = {
    header: "Keep nature thriving",
    body: "Your support protects the places and wildlife we all share.",
    cta: {
      label: "Donate now",
      action: "redirect",
      href: "https://example.org/donation-form", // replace with the donation page URL
    },
    dismissLabel: "No thanks",
    en: { referenceField: "en_txn10" },
    triggers: {
      frequencyDays: 0, // for testing only
      list: [{ type: "exit-intent" }],
    },
  };
</script>
<script src="https://aaf1a18515da0e792f78-c27fdabe952dfc357fe25ebf5c8897ee.ssl.cf5.rackcdn.com/2246/en-lightbox.js" async></script>

```

Page 2 — donation form

The donation page must have an EN form with a field named `en_txn10`, then it can use this minimal config:

```

<script>
  window.ENLightbox = {
    en: { referenceField: "en_txn10" },
  };
</script>
<script src="https://aaf1a18515da0e792f78-c27fdabe952dfc357fe25ebf5c8897ee.ssl.cf5.rackcdn.com/2246/en-lightbox.js" async></script>

```

With no header, button, or triggers, no lightbox appears on page 2; the script only fills the `en_txn10` field.

What happens

1. When the visitor clicks the main button on page 1, the lightbox writes `lightbox_accepted` into `en_txn10` on page 1 and remembers that outcome for the browser session.
2. Page 2 fills its own `en_txn10` field before the visitor touches anything.

3. The value submits with the donation. After the successful replay, the carry-over is cleared, so it is written once on page 2 and then forgotten.

This works within the same browser tab and session. The page 1 EN form must be present for the first write, and the donation page must use the same field name, `en_txn10`. Closing the lightbox with X, Escape, the background, "No thanks," or a secondary close action carries `lightbox_declined` the same way; a main button with `action: "close"` is still treated as accepted.

FAQ

Is the lightbox tracked in Tealium / Adobe Analytics?

The lightbox sends two fixed events through Tealium when `window.utag.link` is available; it does not send directly to Adobe Analytics. See [Analytics tracking](#) for the exact payloads and a way to verify them.

Can the button link to another page?

Yes. Add an `href` to the `cta` object and set `action: "redirect"`. The button becomes a real link — it navigates normally, and visitors can middle-click or right-click it. This works on all trigger types, including exit-intent. It's most useful on time-delayed and scroll-triggered popups.

```
cta: { label: "Donate monthly", href: "https://support.nature.org/...", action: "redirect"
}
```

Can we use any design with any trigger?

Yes. The theme, layout, and image position are completely independent of the trigger. Any design pairs with any trigger.

Can we record lightbox outcomes in an Engaging Networks reference field?

Yes. Configure the reference field designated by Membership:

```
en: { referenceField: "supporter.appealCode" }
```

When the configured name is safe, the library ensures the matching input during listener installation, before any lightbox interaction. It selects the real EN page-builder form first and uses the legacy EN form selector as a fallback. If the form is absent while the page is still loading, it retries once on `DOMContentLoaded`; if the document is already loaded and no form is present, it does nothing.

This empty ensure is not an outcome write: it emits no `enlb:field-write`, does not create pending carry-over storage, and does not affect analytics or frequency state. The primary CTA writes `lightbox_accepted`; close, Escape, overlay, secondary, dismiss, and API close paths write `lightbox_declined`. A primary CTA with `action: "close"` remains accepted. Dotted EN names such as `supporter.appealCode` are supported. The eager ensure leaves an existing same-name input of any type completely unchanged; otherwise it creates one empty, optional hidden input without an `id`. Only a later accept, decline, or replay outcome write updates the input's `.value`; those writes do not change its other attributes.

A redirect CTA carries the latest outcome through the current session only after the origin page wrote it to its EN form, so the origin form must be present. For carry-over, the destination page must embed `dist/en-lightbox.js` and use the same configured field. Replay waits automatically for DOM readiness, so a config-before-script embed in `<head>` is allowed; the value is cleared after a successful replay. Membership owns the deployed field designation—avoid `en_txn2` when the annual upsell uses it, and do not share a field with another campaign.

Can the close button be bigger / easier to see?

Yes. For `light` / `dark` / `brand` it's 44×44 pixels with a contrasting round backing — significantly larger than a standard close button — and it stays visible over photographs and colored backgrounds. The `forest` and `sky` campaign themes use their own close × instead (a square green button for forest, a plain black × for sky). You can place it inside the dialog, outside the dialog, or remove it entirely.

Can visitors on mobile or tablet use it?

Yes. The popup works on desktop, tablet, and mobile. On small screens the two halves stack — with the image always on top (whatever your `imagePosition`) and the close × always carrying a visible backing so it reads over the image. The image shows by default; set `hideImageOnMobile: true` to hide it. The only exception is the exit-intent trigger, which is desktop-only (mobile browsers don't have a mouse cursor to detect).

Does the popup block the donation form?

No. The popup is a non-blocking overlay. It never touches or interferes with any Engaging Networks form on the page. If an error occurs while opening, it simply doesn't open — the page keeps working normally.

How do I update the popup across multiple pages?

Each page has its own copy of the config snippet. To make a change, edit the `window.ENLightbox` block on each page where it appears. Search your Engaging Networks pages for `en-lightbox.js` to find all the pages that currently use it.